

Instantly Shareable Playtest

Implementation Specification: Playtest × xCloud streaming

Melanie Chen · Microsoft Xbox Intern

Draft June 2026

Table of Contents

1. What We Are Building
2. End-to-End Flow
3. Design Decisions & Tradeoffs
4. Team Responsibilities
5. Data Contract
6. REST API Contract
7. Known Blockers
8. Resolved Decisions
9. Open Questions & Recommendations
10. Success Criteria (P0)
11. Milestones & Foundational Demo
12. Glossary — *italicized terms are defined here*

1. What We Are Building

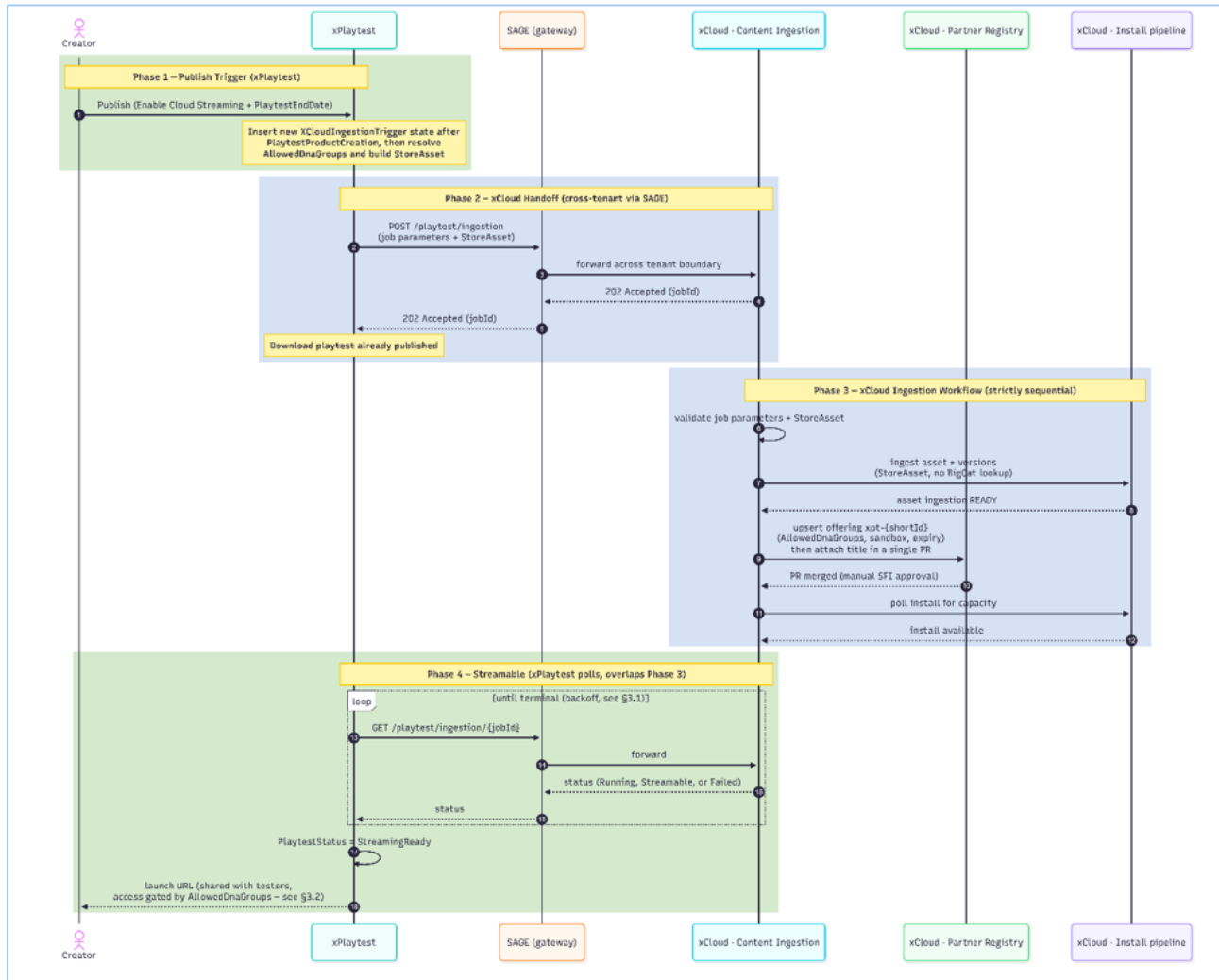
Today, xPlaytest lets game creators distribute private pre-release builds to testers as downloadable installs through Garrison. Modern games are hundreds of gigabytes — downloads are slow, hardware provisioning does not scale, and shipping physical devices to testers risks unreleased code leaking.

The Instantly Shareable Playtest project wires xPlaytest into Xbox Cloud Gaming's streaming pipeline. When a creator publishes a playtest with the **Enable Cloud Streaming** option checked, their RETAIL-signed PC build is ingested into xCloud and becomes streamable to authorized testers via a one-click shareable link — **in addition to** still being available for download.

This is purely additive. Creators who do not opt in get the existing download-only flow. Creators who do opt in get both download and streaming for the same playtest.

2. End-to-End Flow

2.1 Diagram Flow



2.2 Table Flow

A streaming-enabled playtest goes through four phases. The first phase runs entirely on xPlaytest; the middle two run on xCloud; the fourth is a status reconciliation across both.

#	Phase	What Happens	Owner
1	Publish Trigger	Creator clicks Publish in Partner Center. PublishPlaytest validates the playtest and hands off to the existing XPackagePlaytestPublishWorkflow. The workflow runs the existing state machine (FetchingContentIds → StartingContentSubmissionJob → PollingContentSubmissionJob → PlaytestProductCreation). We insert a new state , XCloudIngestionTrigger, immediately after PlaytestProductCreation and before SuccessCompletion. This is the right integration point because the workflow already calls cross-system services (XProduct, SUCU, GMS) and is built for long-running, retrieable execution.	xPlaytest
2	xCloud Handoff	The new state builds a single PlaytestIngestionJobParameters payload (see §5) from data the workflow has already resolved for XProduct. The payload combines the <i>offering</i> metadata (the resolved DNA group IDs) with the <i>ingestion</i> metadata (a fully formed StoreAsset). It POSTs the payload to SAGE, which proxies the call cross-tenant to services.contentingestion. The response is 202 Accepted with a jobId. Ingestion failure does not block the download playtest from publishing.	xPlaytest → xCloud
3	xCloud Ingestion Workflow	A new PlaytestTitleIngestionWorkflow runs inside services.contentingestion. It 1. Validates inbound parameters 2. Chains the existing AssetIngestionWorkflow using the supplied StoreAsset (no XProduct lookup)	xCloud

#	Phase	What Happens	Owner
		- Writes the playtest-specific offering and attaches its title in a single Partner Registry PR - Polls the install pipeline for capacity.	
4	Streamable	When the xCloud workflow reaches a terminal state (Streamable / Failed / InstallNotFound), xPlaytest learns about it by polling the SAGE status endpoint with the jobId. On Streamable, xPlaytest sets PlaytestStatus = StreamingReady (new enum value) and surfaces the launch URL https://play.xbox.com/play/launch/{productId}?offeringId=xpt-{id} in Partner Center.	xCloud → xPlaytest

3. Design Decisions & Tradeoffs

3.0 Streaming requires a resolvable Xbox Title ID

Decision: The "Enable Cloud Streaming" option is offered only when the selected package is Xbox Live-configured — i.e., it has a resolvable numeric Xbox Title ID. This is enforced as a check at publish: if the creator has never published an Xbox Live configuration for the title, streaming is unavailable and the publish proceeds download-only.

Rationale: XboxTitleId is required by the xCloud streaming-ingestion contract for games (as *XCloud's* StoreAsset.Validate() rejects a null/zero value) and *Playtest's*

PlaytestProductDocumentBuilder currently hardcodes it empty, so ingestion would fail validation without a real value. The ID is not minted by Partner Center on publish. For a normal title it is derived from Big Cat, but xPlaytest is the service that populates the playtest's XProduct document, so nothing upstream fills the title ID in for us. It also *can't* be reverse-resolved since XProduct only supports lookup by product Big ID, not by alternate ID and the playtest's own generated product doesn't exist in XProduct until after we write it. The title ID therefore has to be read from the parent/source product. Because our team owns XOrc we can source it there given the parent product ID and XOrc returns the package family name, application ID, and title ID.

Trade-off Accepted: Developers whose title has never been published cannot stream their playtest in v1 they get download-only.

3.1 Manual PR Approval

Constraint: Every Partner Registry write creates an ADO PR. SFI policy forbids the caller from self-approving in production.

v1 mitigation:

1. Create a new Partner Registry endpoint that creates an offering and attaches a title in a single PR instead of two sequential writes.
2. For project scope, we accept that someone on the xCloud side will approve those PRs out-of-band.

v1.1: If the payload carried the parent product ID, an automated pre-approval could match and gate auto-approval that way. This requires an SFI exception and is GSS's call.

3.2 Authorization Gate at the Offering vs Title Level

Decision: Playtest offerings are **private**. Access is gated by a new `AllowedDnaGroups` field on the offering's `PlayerAuthorizationOptions`. The title inside the offering bypasses the per-title entitlement check.

Rationale: The alternative (public offering and gate at title level via package flighting) introduces entitlement checks into session and permissions services by centralizing access control at the offering layer. Gating at the offering level gives us a single, firm `GsToken.DnaGroups` check at offering entry, after which any title in that offering is reachable. This is the pattern `TestX` already uses successfully. One-offering-per-playtest means there is no risk of accidentally exposing an unrelated title.

Tradeoff: Skipping the per-title entitlement check is a slight relaxation of defense-in-depth. The mitigation is that the offering is private and DNA-group-locked, and the playtest title is not catalogued anywhere a non-tester could find it.

External dependency: Partner Registry only stores the allow-list; the actual `GsToken.DnaGroups` ↔ `AllowedDnaGroups` check happens in the streaming-token / Bayside path.

3.3 Pick One DNA group as the Effective "Flight" for Installation

Decision: When the xCloud install pipeline (SUCU/allocator) needs a single flight identifier to resolve the package version, the workflow passes the **lexicographically smallest DNA group GUID** from `AllowedDnaGroups`.

Rationale: A playtest can have multiple DNA groups. Each materializes in SUCU as its own "flight." All DNA groups for a single playtest point to **the same effective build version** and there is no per-group divergence in this scenario. SUCU/the install pipeline expects exactly one flight ID at the lookup site. A deterministic choice (lex-smallest) means re-runs of the workflow with the same input pick the same flight, which is helpful for idempotency and debugging.

Caveat — flight selection is not "sticky": "Lex-smallest of the current set" is computed at every resolve. If the audience changes such that a new GUID is added that sorts lower than the previous smallest (or the previous smallest is removed), the resolved flight id will change for subsequent resolves. This is acceptable v1 behavior because the install pipeline addresses flights by GUID alone and there is no notion of "first-chosen flight."

This is an install-time lookup rule only. It does NOT change audience auth and we still check the user's `GsToken` against the full `AllowedDnaGroups` set.

Tradeoff: If the install pipeline ever needed per-DNA-group version divergence, this rule would have to change. That is not a near-term scenario; the workaround is contained inside `PlaytestTitleIngestionWorkflow` and easy to revisit.

3.4 Streaming is Optional and in Addition to the Standard Playtest

Decision: And, not or. Publishing a streaming-enabled playtest also produces a downloadable playtest in the existing flow. The streaming part is gated behind an “Enable Cloud Streaming” checkbox at publish time.

Rationale: Treating the two as mutually exclusive would surprise creators who already rely on the download flow and would force a hard cut-over. Layering streaming on top keeps the existing behavior unchanged for everyone who doesn’t opt in, and gives streaming users the maximum surface to share their build.

3.5 One playtest ↔ One title ↔ One offering

Decision: Each published playtest gets exactly one xCloud offering with exactly one title. The offering id is `xpt-{shortId(PlaytestId)}`.

Rationale: Each playtest can target a different audience (different DNA group set). Offering ↔ audience must be 1:1 to keep the authorization model crisp. The future plan is to cluster a single developer’s playtests into one offering to reduce catalog noise, but explicitly *not* within this project’s scope. Each playtest is already its own document in XProduct, so a 1:1 mapping on the xCloud side matches the upstream identity model.

Tradeoff: A developer with many active playtests creates many offerings. If this becomes a catalog noise problem, the offering ID scheme is stable enough to evolve later. The xCloud client could implement a “log into all my private offerings” flow without changing the data model.

3.6 PC-first, Xbox Console Deferred

Decision: v1 scope is **PC builds only**.

Rationale: The data path is already set up for PC (`WINDOWS.DESKTOP` platform constant, PC-only `ContainsMSIXVCByDeviceFamily`). PC requires `AumID` and `PackageFamilyName`, both either available or sourceable. Console adds Xbox-specific Title ID and package handling that we don’t need for the foundational integration.

AumID resolution path: `AumID` is only consumed by the server’s upload-login telemetry, so a constant default is acceptable in v1 and unblocks the publish flow. In V1.1, we will plumb the real `ApplicationId` from the AppX manifest as a follow-up.

Risk: PC support in the install poll path: The existing `TitleIngestionWorker.PollFirstInstallAsync` (`TitleIngestionWorker.cs`) hardcodes Xbox-specific allocator parameters:

```
ServerFilter serverFilter = new()
{
    Region = this.environmentProvider.IsProd() ? GsRegionName.WestUs2 : GsRegionName.WestEurope,
    PoolId = ... ? "XBOX_CERT" : "XBOX_MAIN",
    SystemUpdateGroup = SystemUpdateGroup.GA,
    ServerType = ServerType.XboxV3SeriesS, // ← Xbox-only
    LocalPackageId = targetInstallId
};
```

`ServerType.XboxV3SeriesS` and the `XBOX_*` pool ids are console-only. The PC streaming pipeline uses its own pool / server type that this code never references. **XC6 must include an explicit audit + fork of the install poll path so PC playtests pick a PC pool (likely a different `ServerType` value and a PC pool id) before any PC end-to-end test can succeed.** If the PC install path differs more than a per-parameter swap (e.g., entirely different allocator service), XC6 grows in scope and the call site needs a platform-aware branch. This is now tracked as an open item under XC6 and as a row in Known Blockers §7.

Tradeoff accepted: Xbox users can't stream playtests until v1.1. The plumbing for Xbox is incremental (`XboxLiveTitleId` is already a P0 we need anyway), so adding it later is bounded.

3.7 Naming and Identifiers

Decision: Offering id is `xpt-{shortId}` (opaque). Title id is `<alphanumericOnly(PlaytestName)>{PlaytestGeneratedXProductBigId}` (semi-readable, suffixed for uniqueness).

Rationale: An offering id must be ≤ 32 chars and a full GUID is 36 chars; the `xpt-` prefix + GUID would be 40 so we hash/truncate to a short id. Title ID is more visible in some xCloud UX and appending the playtest product ID gives uniqueness for a developer with multiple playtests sharing the same game name. We accept some opacity because mapping schemes that try to be "pretty" would either lose uniqueness or force us to maintain a name-resolution layer.

Tradeoff documented: When a tester or developer sees `xpt-7a3b8f1c...` in any URL or diagnostic, they will not be able to identify the playtest by name. We mitigate by surfacing the friendly playtest name everywhere we control the UX (Partner Center, launch tile), and reserving the opaque id for backend/URL use only.

3.8 Bespoke endpoint vs. generic "CreateOfferingFromStoreData"

Decision: Bespoke `POST /v3/playtest/playtestingestion`.

Rationale: - A generic "CreateOfferingFromStoreData" surface is the future direction (GSS platform team is discussing an orchestration layer on top of their workflow service), but is

explicitly out of scope. The bespoke shape lets us encode playtest-specific defaults (sandbox = RETAIL, isPrivate = true, etc.) on the server side and keep the contract small.

Tradeoff accepted: Future scenarios that resemble playtest may duplicate boilerplate. The bespoke route is small enough that migrating to a generic one later is a controlled refactor.

3.9 Cross-tenant S2S strategy

Short term: xPlaytest Service (MSFTGreen tenant) → xCloud SAGE (Corp tenant) using the Green→Corp cross-tenant auth scheme being added by PR #15715445.

Long term: xCloud is migrating to PME by end of summer. PME does not accept Green callers out of the box. The GSS platform team owns the long-term solution, which is **not** within scope.

3.10 Polling vs Callback for Ingestion Completion

Decision: xPlaytest learns about ingestion completion by polling `GET /v3/playtest/ingestion/{jobId}` on SAGE.

Rationale: SAGE today is a **request/response proxy**. There is no callback/event-hub orchestration layer. Adding one is a non-trivial gateway change that does not fit summer scope. xPlaytest already polls inside its own workflow and we can reuse the existing workflow framework. `services.contentingestion` already exposes job status through `WorkflowsControllerV3`; SAGE just needs to forward.

Polling strategy: FINALIZE THIS SECTION

3.11 API versioning strategy

Decision: Two layers, two conventions.

Layer	Routes	Versioning convention	Why
External (xPlaytest → SAGE)	POST /playtest/ingestion, GET /playtest/ingestion/{jobId}, DELETE /playtest/ingestion/by-playtest/{playtestId}	Unversioned today; versioned via OpenAPI info.version (1.0.0)	SAGE's existing public route style is unversioned. We treat the OpenAPI document as the authoritative versioned contract and rely on additive evolution (new optional fields), reserving a future <code>/v2/playtest/ingestion</code>

Layer	Routes	Versioning convention	Why
			prefix only for breaking changes.
Internal (SAGE → services.contentingestion)	POST /v3/workflows/playtestingestion, GET /v3/workflows/playtestingestion/{jobId}, DELETE /v3/workflows/playtestingestion/by-playtest/{playtestId}	URL-versioned (/v3/...)	Matches the existing WorkflowsControllerV3 convention. Downstream callers are all in-tenant and already follow this pattern.

Evolution rules — both layers: - Prefer **additive optional fields** over breaking required ones. - Document required vs. optional explicitly in `openapi/playtest-ingestion.yaml` (the OpenAPI doc covers the external SAGE layer; the internal layer is documented inline in `services.contentingestion`). - A breaking change to the external surface bumps to `/v2/playtest/ingestion` and is supported in parallel for at least one xPlaytest release.

Specifically expected future additions (not breaking): `CallbackUrl` once cross-tenant messaging exists (replaces polling)

3.12 Streamable Playtest End Time

Constraint: Streaming-enabled playtests must have an explicit `PlaytestEndDate` rather than the platform default of `DateTime.MaxValue` (no expiration). Download-only playtests are unaffected and keep the optional, no-expiry default.

Rationale: A streaming offering is live infrastructure and for as long as it exists it ties up xCloud streaming capacity and stays as a live entry in their catalog. With no end date there's no natural signal to ever tear it down, so an open-ended streaming playtest lingers indefinitely and accumulates. A bounded `PlaytestEndDate` lets the offering auto-expire and be reclaimed once the test window closes.

Tradeoff: Diverges from the download flow (which can run indefinitely) and adds one required field for streaming creators

4. Team Responsibilities

4.1 xPlaytest deliverables

Note: The table is presented in **call sequence** order

ID	Task	Notes	SWAG (days)
PT1	Integrate xCloud trigger into XPackagePlaytestPublishWorkflow	New XCloudIngestionTriggerState between PlaytestProductCreation and SuccessCompletion. Reuses existing workflow orchestration + retry policy. Failure does NOT block publish. Also: streaming-publish validators (a) require PlaytestEndDate to be set when streaming is on (b) seller / title allow-list gate	6
PT2	Construct StoreAsset from Playtest product/package data	By the time the new xCloud-trigger state runs, PlaytestLifecycleState is fully populated (FetchContentIds resolved ProductId / ContentId / PackageFamilyName / XProductBigId; PollContentSubmission confirmed they are committed in Partner Center), so most of the payload is a direct projection of in-memory state — no new external calls. Two pieces are <i>not</i> present today: (1) XboxTitleId resolver (P0 cross-team blocker) replacing the XboxLiveTitleId = string.Empty hardcode at PlaytestProductDocumentBuilder.cs:53–70 by reading alternatelds[XboxTitleId] from the XProduct response; (2) AumID default-or-resolver — per Timi the server uses AumID only for ingestion telemetry, so the constant "{packageFamilyName}!App" is acceptable for v1 (v1.1 plumbs the real ApplicationId from the AppX manifest).	4
PT3	Build offering config from audience data	DNA group IDs already resolved via GMS (PlaytestPublishJobParameters.FlightIds). Just package into payload.	1
PT4	Send ingestion request via SAGE	New XCloudIngestionClient (no equivalent today). Cross-tenant S2S token acquisition. 202 Accepted + jobId handling, persistence of jobId on PublishedPlaytestEntity. Blocked on SAGE allow-list (XC2).	3
PT5	Handle completion signal	Polling loop with exponential backoff (§3.1) + background reconciliation job for slow ingestions past the 60-min foreground deadline. State transition to StreamingReady. New launch URL builder. New StreamingReady value in PlaytestStatus enum.	6

ID	Task	Notes	SWAG (days)
PT6	Handle lifecycle updates	Re-trigger ingestion on audience change, expiry change, build republish; teardown on delete via the new SAGE DELETE /playtest/ingestion/by-playtest/{playtestId} route. Hooks into existing UpdatePlaytestAsync (PlaytestBusinessLogic.cs:408-553) and delete workflow.	5
	xPlaytest total		25

4.2 xCloud deliverables

ID	Service	Task	Notes	SWAG (days)
XC1	SAGE	Add new routes: POST /playtest/ingestion, GET /playtest/ingestion/{jobId}, DELETE /playtest/ingestion/by-playtest/{playtestId}	Config-driven in appsettings.xcloud.json. No C# change. Returns whatever services.contentingestion returns (so 202 contract lives downstream).	3
XC2	SAGE	Allow-list the Playtest Service S2S identity	AuthorizedClientAppIds + UseCrossTenantAuth = true on both new routes. Depends on cross-tenant scheme being live.	1
XC3	services.contentingestion	Implement PlaytestTitleIngestionWorkflow	New sealed workflow class. Stages: validate → schedule asset ingestion → poll → upsert offering (via partner registry HTTP client) → attach title → poll install readiness → emit terminal state. Also new contract type PlaytestIngestionJobParameters, WorkflowsControllerV3.Playtes	9 (split into 4 tasks ≤ 5d each in ADO breakdown)

ID	Service	Task	Notes	SWAG (days)
			tlIngestion POST/GET routes, and the DELETE .../by-playtest/{playtestId} tombstone route used by PT6.d.	
XC4	services.partnerregistry	Extend PlayerAuthorizationOptions with AllowedDnaGroups + AllowedSandboxId	New fields on OfferingV2 contract. JSON schema migration for stored offering documents (storage is git/ADO, not SQL). Also: bulk-edit support (combine create-offering + attach-title in one PR to satisfy SFI manual-approval policy) and content-ingestion SPN allow-listing on Partner Registry write routes.	5
XC5	User Login / Offering Auth path	Add DNA-group check to offering access	GsToken.DnaGroups $\supseteq$ at least one of offering.AuthorizationOptions.AllowedDnaGroups $\rightarrow$ grant; else deny. For playtest titles, bypass the per-title entitlement check. Lives outside the four repos in the streaming-token / Bayside path.	5
XC6	Install pipeline	Single-flight-id selection from DNA groups + PC fork	Pick lex-smallest GUID; audit TitleIngestionWorker.cs:197-225 install path to confirm tolerance of non-GA flight ID; branch ServerFilter build on Platform so PC playtests	4

ID	Service	Task	Notes	SWAG (days)
			don't fall into the Xbox-only allocator parameters (see §3.6 risk callout).	
	xCloud total			27

5. Data Contract

5.1 PlaytestIngestionJobParameters (xPlaytest builds, xCloud consumes)

Field	Type	Source on xPlaytest	Required	Notes
PlaytestId	Guid	PartnerCenterProductId for stable offering identity	yes	Used to derive OfferingId = xpt-{shortId(PlaytestId)}
AllowedDnaGroups	string[]	PlaytestPublishJobParameters.FlightIds (resolved via GMS)	yes	xCloud wraps each in XusAudience(g, RetailSandboxId)
ExpirationTime	DateTime	PlaytestPublishJobParameters.PlaytestEndDate	yes when IsStreamingEnabled = true	Per the June 3 sync, a streaming-enabled playtest must carry an explicit end date. Download-only playtests are unaffected. No hard cap; soft warning at 180d (see §9 Q3).
AllowedSandboxId	string	PackageConstants.RetailSandbox = "RETAIL"	no	Defaults to RETAIL server-side — passing is

Field	Type	Source on xPlaytest	Required	Notes
				allowed but discouraged
StoreAsset	object	See §5.2	yes	Existing xCloud contract StoreAsset.cs:17
IsGreenSigned	bool	PlaytestPublishJobParameters.IsGreenSigned	no	Defaults to true (all playtests are green-signed today)
CallbackUrl	Uri	n/a	no	Reserved , not used in v1. v1.1 hook for callback delivery once cross-tenant messaging exists.
CorrelationId	string	xPlaytest-supplied trace id	no	For diagnostics; xCloud echoes in status response

5.2 StoreAsset (xPlaytest fills)

This is the existing contract at StoreAsset.cs:17 — we do not redefine it. xPlaytest must produce values that pass StoreAsset.Validate() (StoreAsset.cs:25-61).

Field	Source	Notes
StoreEntry.ProductId	PlaytestPublishJobParameters.PlaytestGeneratedXProductBigId	Confirmed in meeting (Anthony, 14:00). Big Cat lookup is not done from this id.
StoreEntry.Flags	const GAME	
StoreEntry.Name	PlaytestResponse.Name	
StoreEntry.Description	PlaytestResponse.Description	

Field	Source	Notes
StoreEntry.PublisherId	PublishedPlaytestEntity.SellerId	
StoreEntry.IsPrivateAudience	true	All playtests are private
StoreEntry.ParentProductIds	[]	Playtests have no parent-product hierarchy today
Markets	[Id.Parse("US")]	Single-market for playtest publish path
Platform	"WINDOWS.DESKTOP"	PC-first (§3.6)
ContentId	PlaytestLifecycleState.PlaytestContentId	In the XProduct doc this corresponds to <code>packages.{packageId}\{versionId}.contentId</code> . For a playtest product the published XProduct typically contains multiple package entries, some with null contentId (verified against <code>Files/product_testproduct0001_xproduct-document_primary.json</code>). xPlaytest selects the package using these rules: (1) filter packages to entries whose platform matches the publish platform (<code>v1 = WINDOWS.DESKTOP</code>); (2) keep only entries with state $\in \{\text{Published, Active}\}$ (skip Draft/Pending); (3) prefer entries with a non-null contentId; (4) if multiple remain, choose the most recently uploaded by <code>packageVersionUploadTime</code> . If contentId is still null after selection, fall back to <code>servicingContentId</code> (also on the <code>packages.{packageId}\{versionId}</code> object) — both fields are produced by the same downstream BigCat pipeline and the playtest workflow accepts either. The selected value must equal <code>SourceId</code> derived inside the workflow.
XboxTitleId	XProduct <code>alternatIds[idType=XboxTitleId].value</code> (verified path in <code>Files/product_testproduct0001_xproduct-</code>	P0 blocker — validator throws if null/0 for games. <code>PlaytestProductDocumentBuilder.cs:53-70</code> currently hardcodes empty.

Field	Source	Notes
	document_primary.json; string-typed in the doc, cast to uint)	
PackageFamilyName	XProduct properties.packageFamilyName (verified). Already surfaced today as PlaytestLifecycleState.PackageFamilyName.	Required for PC
AumID	v1: constant default acceptable per Timi (meeting 20:11). v1.1: {PackageFamilyName}!{ApplicationId} with ApplicationId from XProduct / AppX manifest.	Validator throws on null for PC games.

5.3 Derived inside `PlaytestTitleIngestionWorkflow` (xPlaytest does NOT send)

Field	Computation
PartnerId	Id.Parse("PLAYTEST") (per Timi/David comment thread — keep namespacing simple)
OfferingId	\$"xpt-{ToShortId(PlaytestId)}", must be ≤ 32 chars
TitleId	<alphanumOnly(StoreAsset.StoreEntry.Name)><PlaytestGeneratedXProductBigId> (resolves comment TB65). Supersedes ARCHITECTURE.md §2 / §4.3 which previously documented TitleId = OfferingId as a placeholder. The two values are intentionally distinct: OfferingId is the catalog/auth key (opaque, short); TitleId is the install/streaming key (user-readable, append-style uniqueness per Anthony). Both are persisted on PublishedPlaytestEntity so the xPlaytest side can refer to either.
SourceId	StoreAsset.ContentId
PackageNameOverride	StoreAsset.StoreEntry.Name (display name for the offering)

Field	Computation
Audiences	AllowedDnaGroups.Select(g => new XusAudience(g, XusAudience.RetailSandboxId))
AssetId	Generated by AssetIngestionWorkflow.CreateAssetAsync
StreamingPackageIds	From child AssetVersionIngestion jobs (SUCU query using SourceId + Audiences)
Effective flight id for SUCU	Lex-smallest GUID from AllowedDnaGroups (§3.3)

6. REST API Contract

6.1 POST /playtest/ingestion

Triggers asynchronous playtest ingestion. Returns immediately with a jobId.

- **Auth:** S2S bearer, cross-tenant. Caller SPN must be in SAGE's AuthorizedClientAppls for this route.
- **Request body:** PlaytestIngestionJobParameters (§5.1).
- **Response:** 202 Accepted with { jobId: string, statusUrl: string }.
- **Error codes:**
 - 400 invalid StoreAsset (validator rejection) — body includes field-level errors
 - 401/403 auth failure
 - 409 duplicate PlaytestId already in flight in a non-terminal state — response body (JobConflict schema) includes the existing jobId and statusUrl so the caller rejoins polling instead of starting a fresh job
 - 5xx retryable; caller should requeue via the existing workflow retry policy

6.2 GET /playtest/ingestion/{jobId}

Polls workflow state. Caller uses the exponential backoff from §3.10.

- **Auth:** Same S2S bearer as POST.
- **Response 200:** { jobId, status, terminalState?, errors[], updatedAt, correlationId? } where status ∈ { Running, Streamable, Failed, InstallNotFound }. terminalState is set when status is terminal.
- **Response 404:** Unknown jobId.

6.2.1 DELETE /playtest/ingestion/by-playtest/{playtestId}

Removes a published playtest's xCloud offering

- **Auth:** Same S2S bearer.
- **Behavior:** Sets `offering.PlayerAuthorizationOptions.AllowedDnaGroups = []`
- **Response 200:** { `playtestId`, `offeringId`, `tombstoneTime` }. Idempotent: re-deleting an already-tombstoned offering still returns 200.
- **Response 404:** No offering exists for this `playtestId` (never ingested).

Routing note: These three paths are the **external** SAGE routes that xPlaytest calls. SAGE forwards them to `services.contentingestion's /v3/workflows/playtestingestion/{jobId}` and the new `/v3/workflows/playtestingestion/by-playtest/{playtestId}`. Treating the downstream `/v3/workflows/...` paths as an implementation detail keeps the external surface area stable when content-ingestion's internal versioning changes.

7. Known Blockers

Blocker	Problem	Solution
Cross-tenant S2S	Playtest Service runs in MSFTGreen tenant. xCloud SAGE does not allow for calls from MSFTGreen.	Short term: Use PR #15715445 adding Green→Corp cross-tenant auth. In the long term, xCloud will migrate to PME and Green->Corp is not feasible.
Partner Registry PR approvals	Each PR Registry write creates an ADO PR. SFI forbids self-approval. Naïve flow = 2 PRs per publish, both blocking.	Combine offering write + title attach into a single PR via BulkEditAsync (RegistryProcessor{T}). Accept manual approval out-of-band during internship; long-term automation needs an SFI exception.
XboxTitleId source	PlaytestProductDocumentBuild er.cs:53-70 hardcodes empty; validator rejects null/0 for games.	New resolver pulls from XProduct alternatelds[idType=XboxTitleId].value.
AumID for PC	Validator rejects null AumID on PC games. ApplicationId not currently propagated into the publish path.	v1: ship with a constant default (Timi confirmed telemetry-only server side). v1.1: plumb real ApplicationId from AppX manifest.

10. Success Criteria (P0)

11. Milestones & Foundational Demo

1. Foundational Work (target ≤ 6/7)

- XC4 shipped: AllowedDnaGroups writable on OfferingV2.
- XC5 shipped: GsToken DNA-group check enforced at offering entry.
- Manual operator script creates an offering with a fixed DNA-group GUID and attaches an existing test title.
- A Partner Center customer-group membership change for a tester is visible in xCloud access within the GMS propagation latency.

- **Acceptance:** a tester in the group can stream the test title; a tester removed from the group is denied.

2. Payload construction: StoreAsset & offering info (target ≤ 6/14)

- PT2 + PT3 shipped: from an existing playtest's in-memory state, the builder produces a valid StoreAsset (incl. resolved XboxTitleId) and the corresponding offering/AllowedDnaGroups config.
- **Acceptance:** for a real playtest product, the builder emits a fully-populated, validator-passing PlaytestIngestionJobParameters payload (verified offline, no SAGE call yet)

3. SAGE connectivity (target ≤ 6/30)

- XC1 + XC2 + XC3 + XC6 shipped: a manual curl against SAGE with the payload from Milestone 2 succeeds end-to-end and produces a streamable offering.
- Proves cross-tenant S2S token acquisition and the SAGE allow-list path.
- **This is the Midpoint Connect demo:** end-to-end ingestion driven from a script, even if the Partner Center UI integration isn't wired in yet.

4. Ingestion workflow integration (target ≤ 7/14)

- PT1 + PT4 + PT5 shipped: the XCloudIngestionTriggerState is wired into XPackagePlaytestPublishWorkflow, so publishing a streaming-enabled playtest in Partner Center automatically builds the payload, calls SAGE, and returns a launch URL end-to-end.

5. Lifecycle (target ≤ 7/21)

- PT6 shipped: audience / expiry / republish / delete all reflect correctly on the xCloud side

6. Soak + handoff (target ≤ 7/28)

- OpenAPI / partner docs published.
- Runbook for cross-tenant token rotation and PR-approval triage.

6. Presentation week

- Intern presentation and demo

12. Next Steps

These are the natural add-ons once the v1 foundational integration lands. Items are grouped by theme; several are already referenced inline earlier in the spec.

12.1 UI / Frontend work (Bayside / Edgewater / Partner Center)

- Wire the URL into the Bayside offering-access path, so an authenticated tester who clicks the link is dropped directly into the playtest offering page with no separate sign-in friction.
- Have GSSV return the set of playtest offerings a signed-in user is entitled to, so the frontend can list "playtests available to you" instead of requiring a shared link. Depends on: the offering-level DNA-group auth model (§3.2) and a discovery query over private offerings.
- Add the "Enable Cloud Streaming" affordances in Partner Center: the streaming checkbox (enabled for XboxLive Titles only), the forced end-date picker (pre-filled 90d, §9 Q3), and a DNA-group picker instead of raw GUID entry (see §12.6).

12.2 Authorization & PR-approval automation

- Automated pre-approval via parent product ID. Today every Partner Registry write is a manual ADO PR (§3.10). If the ingestion payload carries the parent product ID, an automated approver could match and gate auto-approval on that, removing the human round-trip.

12.3 Title eligibility: allow non-Xbox-Live titles

- Unblock not-yet-published titles (§3.0 / §3.12). v1 gates streaming on a resolvable XboxTitleId, which a brand-new playtest title doesn't have until it goes live.

12.4 Cross-tenant identity & PME migration

v1 uses the MSFTGreen → Corp cross-tenant schem, but xCloud is migrating to PME by August 2026, and PME does not accept MSFTGreen callers out of the box. Once the migration completes, design and implement the Green → PME cross-tenant auth (likely a Corp/PME proxy, mirroring the Greenbelt pattern).

12.5 Completion delivery — callbacks over polling

Replace polling with a callback (v1.1). v1 polls SAGE for terminal state (§3.1) because there's no cross-tenant message bus today. Once the GSS platform team ships cross-tenant messaging, switch to a CallbackUrl-driven completion signal, eliminating poll load on

SAGE/services.contentingestion and the up-to-5-min "ready" latency. Depends on: cross-tenant messaging existing.

12.6 Catalog & clustering

- Right now (v1), every playtest is completely separate: **one playtest = one title = one offering** (§3.5). For a developer who runs lots of playtests, this creates a huge pile of separate offerings cluttering the catalog. We should look into bundling all of a seller's playtests together so a tester could "log into all of this seller's private playtests" in one place instead of hunting through many. Can't start this until we have a reliable way to list all the groups a seller owns.
- Use Brian's "seller's audience groups" API (§9 Q6). Brian has an API (GetGroupsBySellerAsync) that lists the audience groups belonging to a seller. We'd use it for letting a developer pick which group can access a playtest when they set it up

12.7 Platform expansion

Xbox console support (Garrison / Bastion). v1 is PC-only (§3.6). Extend to console, which adds Xbox-specific Title ID + package handling and forks the install-poll path so console picks console pools/ServerType (the inverse of the PC fork tracked in XC6). Also covers launch arguments, region / touch-control configuration, and in-stream feedback collection (project-description P3). Depends on: the install-poll platform branch.

Real AumID plumbing (v1.1). Replace the constant "{packageFamilyName}!App" default with the actual ApplicationId read from the AppX manifest (telemetry-only today per Timi, so non-blocking but worth doing for correctness).

Appendix A: Glossary

- **DNA group:** GMS audience grouping; the identifier we resolve to from Partner Center customer groups.
- **GMS:** Group Management Service. Resolves audience → FlightIds (which in this codebase are DNA group GUIDs, despite the name).
- **GsToken:** Game Streaming token; carries DnaGroups claims for the signed-in user.
- **SAGE: Service API Gateway;** xCloud's cross-tenant ingress proxy. It is the single, allow-listed entry point through which external callers (like xPlaytest, in the MSFTGreen tenant) reach xCloud services in the Corp/PME tenant. SAGE terminates the S2S auth, checks the caller's SPN against the per-route AuthorizedClientAppIds allow-list, and forwards the request to the internal service (e.g., services.contentingestion). xPlaytest never calls an xCloud service directly — every cross-tenant hop goes through SAGE.

- **Ingestion:** An overloaded word; in this spec it specifically means xCloud's process of taking a published build and the metadata payload (PlaytestIngestionJobParameters / StoreAsset) and making the title streamable: validating the payload, registering the asset and its package versions in the streaming catalog (SUCU), creating/attaching the offering, and driving the first install onto a streaming pool. It does not mean Partner Center publish, the xPlaytest download flow, or generic data import — those are separate steps that happen before or alongside it.
- **SUCU:** Streaming Update Catalog; xCloud's package-version resolution layer.
- **SFI:** Secure Future Initiative; the policy regime that forbids PR self-approval.
- **xpt-{id}** — Offering id prefix for playtest-derived offerings.

